

**How to Classify The Degree of A Movie's
Success: Utilizing ML Techniques To Analyze
The Relationship Between Finances and
Popularity**

Charles Diggs

I. Introduction

Entertainment is a form of media that is meant to bring fulfillment to people's lives, which can be demonstrated by television, cinema, etc. With cinema, a film's success can be described through commercial, critical, or cultural impact on society. The commercial success of movies is seen through the box office sales that surpasses the movie's original budget. For example, The Pixar film Toy Story had a budget of \$30 million and generated a revenue of \$401.1 million (IMDbPro 1). In a critical sense, there are aggregate sites such as Metacritic that track certified critic and user reviews for movies, music and more for a score of either a 100 (for critics) and a 10.0 (for users). The cultural impact is defined in a more subjective sense as the public can see its legacy through how the movie is parodied through shows such as Family Guy and South Park, the characters that are used for halloween costumes, and specifically expanding on its legacy through a sequel or a live-action remake.

You can find my code on Github here: <https://github.com/charnerd/ITSC-3156-Final-Project.git>

Motivation

In this project, I wanted to classify the degree of success that a film can have through specific features. I was inspired to do this mostly because I have seen many sequels of old films that date back to the 2000s. For example, the sequels of The Incredibles 2, Zootopia 2, Frozen 2 have been released in recent years and there are current remakes that are in production such as Tangled or the upcoming live-action Zelda movie. Many directors and producers look back at cinema in the past and decide to put a new twist, and their reasoning could be due to the commercial or critical success of the first film. The one thing that intrigues me the most is how what goes into the remake even after the film gets a green light (able to go into production) from

a publishing/production company. Arts and Education Professor Muhammad Fithratullah (Universitas Teknokrat Indonesia) concedes that remakes can compromise elements of the first movie, consider societal demands, and negatively impact the authenticity of what made the original film great (1). This motivation allowed me to begin the process of what methods to apply for solving the mystery of how some motion pictures get greenlit for another part added to their franchise unlike others.

Approach

Initially, this project uses data analysis and preprocessing on the dataset to standardize features and omit unnecessary data. This helps to put a better focus on the certain features of the dataset that can support the problem statement. To predict the outcome of a movie's success by using certain factors such as budget, revenue, and popularity. The prediction of this success is that I will see a high variability in the spread considering the sample set is over 1,000,000 entries. The success class of low, medium, and high is better to understand through the visualizations considering the factors of finances and popularity is what production companies will have to decide on for making sequels and remakes.

II. Data

Data Introduction

My data was retrieved from a dataset that is publicly available on the Kaggle Platform. The dataset is titled “Full TMDb Movies Dataset 2024” (1M Movies) and it contains various data about movies from TMDb (The Movie Database) that is updated daily. Before I utilize preprocessing, the user “asnaiczka” created the list which contains over 1,000,000 samples and

has 24 attributes (features) involved for the film information (1). Please note that the .csv file used for this project was too big to upload to the GitHub repository (more than 500MB), so the link can be found in the README md part of the repository. The table coming up lists all the columns in the database that were originally in the dataset before preprocessing is used to omit unnecessary data and standardize some of the data along with creating the success classes.

Column	Description	Type
id	Unique id	Integer
title	name	String
vote_average	Average vote/rating given by viewers	Float
vote_count	Total count of viewer votes	Integer
status	If a movie is released, rumored, post production, etc.	String
release_date	When the movie was released	String
revenue	Money generated from movie (approximated box office sales)	Integer
runtime	Movie duration (in minutes)	Integer
adult	If movie is suitable for adult audiences	Boolean
backdrop_path	Url for image of movie	String
budget	Budget used for the movie	Integer
homepage	Homepage URL of movie website	string
imdb_id	Movie ID on IMDb site	string

original_language	Language the movie was produced in	String
original_title	Original name for movie	string
overview	Brief summary	string
popularity	Popularity score of movie among users	float
poster_path	Url of movie poster image	string
tagline	Catchphrase/memorable line from movie	string
genres	List of genres the movie applies to	string
production_companies	Companies involved in the film production	string
production_countries	Countries involved in film production	string
spoken_language	Languages spoken in film	string
keywords	Keywords related to the movie	string

The 24 columns and the 1,000,000 samples allows for an extensive analysis on the relationship of finances and the vote average score that will soon fall into one of the success classes. The visualizations will support this data and begin to provide more information for the films that exist in this TMDB dataset.

Preprocessing

The first approach used for preprocessing was to create a function that defines the low, medium, and high success class. The low class score has the factors of the return on investment (ROI) as being less than 1.33 percent, a rating score between 0.0 and 7.7, and a weighted combination of 60% of the ROI and 40% of the rating to be of a score between 0 and 50. An entry of the medium class will have a ROI of greater than 1.33 but less than or equal to 1.67, a rating score between 7.8 and 8.8, and a weighted score in the 50.1-70 range. The high success class has an ROI greater than 1.67, a rating score between 8.9 and 10.0, and a weighted score between 70.1 and 100.

The second approach included is the column removal where a good number of the columns were dropped because they are not as applicable to this project unlike the ones I kept. The columns dropped are either not predictive, sparse, irrelevant, need natural language processing, file paths, or inconsistent formatting. The columns dropped include: id, adult, keywords, backdrop_path, homepage, imdb_id, original_title, overview, poster_path, tagline, and production_companies. The popularity column was added for a bit but I decided to rely on the vote average and count as an alternative to assist in the definition of the success class.

The final approach that was very significant was to remove specific entries from the list for the columns. The rationale behind this is due to a number of missing values for entries for the most crucial columns (budget, revenue, and vote_average). With this entry cleaning, the idea was to remove rows where all of the columns included in the crucial columns are 0. Additionally, if any of the columns used for the success class have invalid values then the entire row is removed. Although, there are still entries that have 0s in some of the columns so the data can mix up and

alter the defining factor for the success class. The biggest reasoning for this can be attributed to the idea that the information is unavailable on the TMDb site.

With these preprocessing approaches, the dataset is catered down to 11,156 samples and 15 columns that account for the features. Through preprocessing, it is clear that a majority of the information is not applicable to my parameters but it can still be useful for other projects.

Visuals

The visualizations for the project are listed below and the analysis is included throughout the figures to make the descriptions more accessible to the graphs. To note, a few visualizations are omitted from this report because the information is repetitive with the graphs I chose for the project.

Figure 1

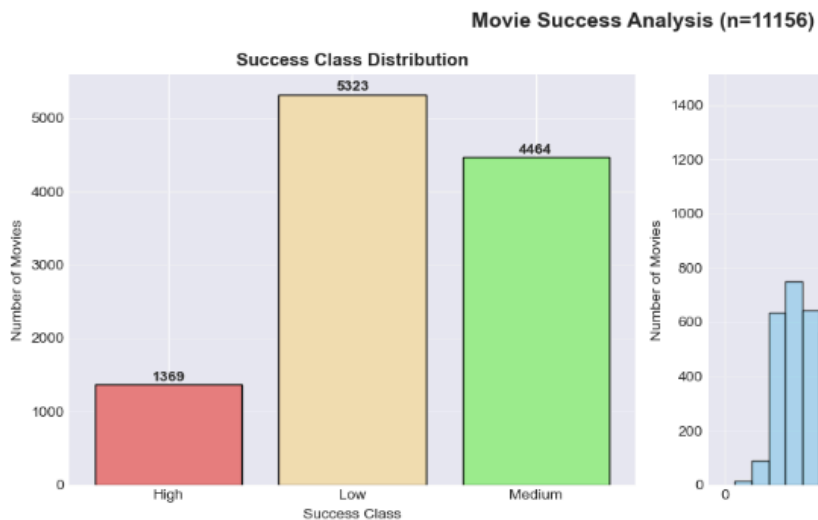


Figure 2

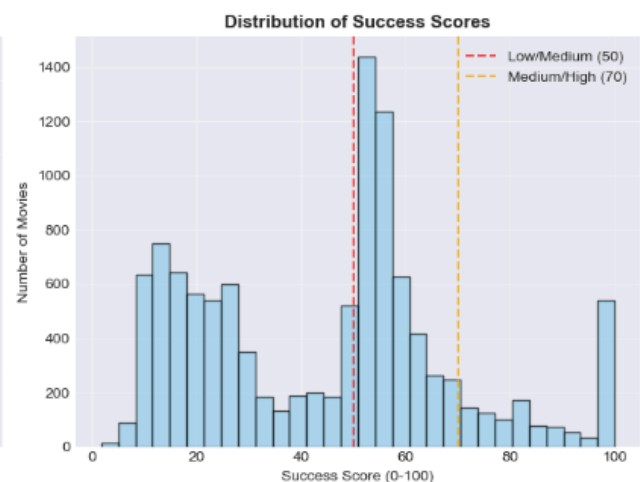


Figure 1: counts the number of movies that fall into each category of success

Figure 2: A set for the success scores that show the frequency of scores

Figure 1 and Figure 2 serve as an initial guide for frequency among the film dataset as seen through the success class and scores. The highest percentage for the class is low while the lowest is the high class, but this may be due to the parameters I set for the classes. The distribution for the bar chart is minimal in variability but it begins to paint a picture that is explained more through Figure 2. The distribution of the classes within the distribution set shows a better variability and there is a higher frequency in the success scores of between 50 and 70 at least for the medium success class.

Figure 3

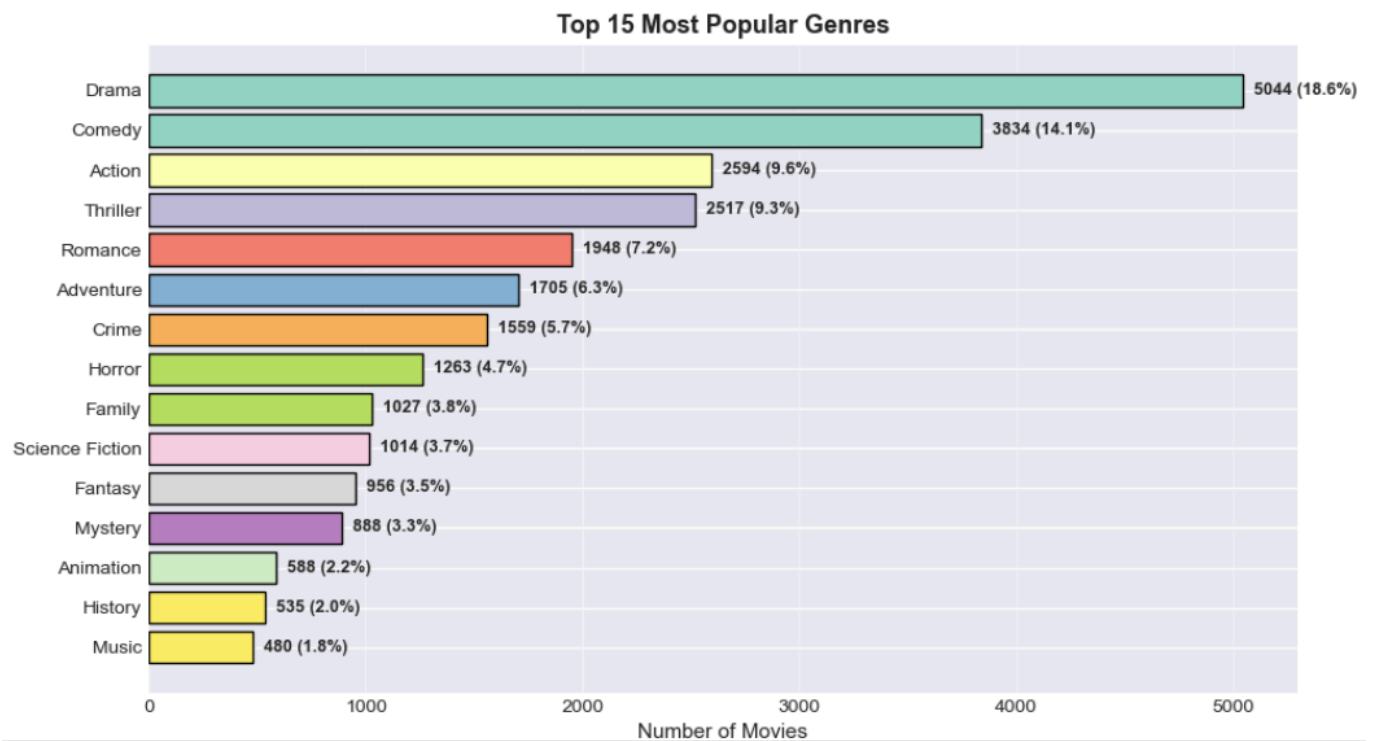


Figure 3: ranks the top 15 most popular genres in the dataset

Figure 3 showcases the top three genres as Drama (18.6%), Comedy (14.1%), and Action (9.6%). Conversely, the least popular genres have Animation (2.2%), History (2.0%), and Music (1.8%). The popularity of these genres is not significant yet but it shows what movies in the TMDB database can account for the voter average and count. As these genres account for a bigger percentage of the dataset, it can prove to be helpful in the variability that was seen in Figure 2.

Figure 4

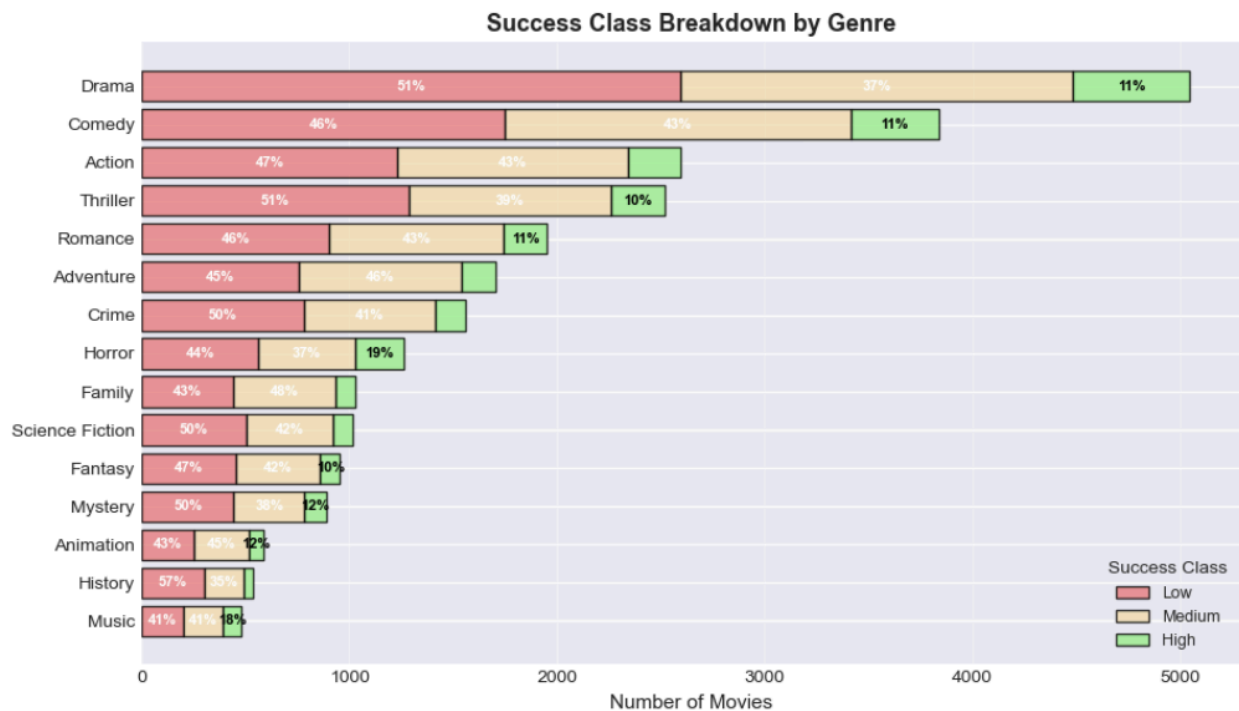


Figure 4: explains the success class breakdown through the genres

Figure 4 continues from the information in Figure 3 and showcases the variability for the success classes. For the low class, the top genres include History (57%), Drama and Thriller (51%) and Science Fiction, Crime, and Mystery (50%). The medium class has the highest genres being Adventure and Family (48%), Action (46%), and Animation (45%). Lastly, the high class shows Horror (19%), Music (18%), and Animation and Mystery (12%) for the top three genres. With this figure, the class breakdown is confusing because a few of the least popular genres have a lower number for the dataset but the percentage is marked nonetheless.

Table 1

Decade	Movies	High%	Medium%	Low%	Avg Score
1950s	191	18.8	52.9	28.3	55.2
1960s	225	20.9	47.6	31.6	54.7
1970s	337	34.4	42.1	23.4	62.8
1980s	873	11.9	43.3	44.8	45.4
1990s	1427	7.4	38.0	54.6	39.7
2000s	2705	6.8	38.6	54.6	39.6
2010s	3615	10.9	41.2	47.9	43.7
2020s	1202	20.7	32.0	47.3	49.4

This table represents the decades that are a part of the dataset. As seen in this figure, the 2010s, 2000s, and 1990s have the most movies but they are ranked low regarding the success class they are in. For the highest success class, the 1970s, 1960s, and 2020s are the highest

percentages. The medium class has the top decade being the 1950s, 1960s, and 1980s. With Table 1, I believe that this spread supports a great variability among the movies and their respective decades. The importance of this variability also means that the Kaggle user has a low bias for movies and pulls them from the vintage, retro, and modern era to create a comprehensive database.

III. Methods

Logistic Regression

Logistic regression is a classification algorithm that models the chances of a movie belonging to a specific success class based on its feature. This classification technique relies on statistics and allows for features to affect success probability. The regression model helps with the interpretability of the success class features but it also helps with predictions of the movie predictions. The interpretability is useful through the features and it can affect the overall success score depending on how high or low the main columns such as ROI, budget, etc. This would be mostly helpful through the classification techniques that fall into the success classes.

Random Forest

The random forest algorithm is used through ensemble learning that builds many trees and combines their predictions. The step-by-step process starts with creating at least 100 decision trees with each tree having a random subset of movies and features. Within this random subset, each tree creates its prediction before the final prediction is defined through the majority vote of all the decision trees. I decided to use this for the algorithm to handle complex and non-linear

relationships since the films in the database can have an extensive success pattern. Furthermore, the random forest technique ultimately has the majority vote correcting errors that exist in the outliers and noise. To add to the reasoning, the algorithm shows a more comprehensive feature landscape through feature importance and it handles mixed data types (integer, float) from the dataset.

IV. Results and Analysis

The logistic regression model pipeline first focuses on the `multi_class` variable is set as multinomial to create separate models internally to create a final prediction for the success class with the highest probability. Additionally, the class weight is set as balanced so the model automatically weights classes that are inversely proportional to frequency. The reasoning is that the high success movies get a higher weight score and the model learns all the classes equally. One other problem that can occur is overfitting, so this is solved through using L2 regularization ($C=1.0$). The regularization is to prevent over-reliance on features and can soon provide business insight for companies to maximize on their high success probabilities.

The random forest model pipeline is used in this project through initializing the `N_estimators` variable to 100 which helps to reduce variance. In other terms, overfitting is another issue that makes the model very complex instead of simpler. My solution approach was to limit the tree depth some more; setting the number to 10 allows the model to capture patterns that relate to defining the success class. Next, it is time to ensure that rules are created from significant patterns instead of tiny samples. The variables that are used to set the numbers to 10

and 5 respectively are `min_samples_split` and `min_samples_leaf`. Finally, the last approach is to use parallel processing for training trees simultaneously. The idea is to set the `n_jobs` variable to 1 because my dataset and features are large.

Table 2

Logistic Regression	Random Forest
Training Time: 27.15 seconds	Training Time: 8.82 seconds
Test Accuracy: 0.969	Test Accuracy: 0.873
Train Accuracy: 0.999	Train Accuracy: 0.880
Test F1-Score: 0.969	Test F1-Score: 0.876

Table 2: provides information on training and testing data scores for the ML algorithms Logistic Regression and Random Forest

The scores table comes from creating the model pipeline to analyze the relationship in the film dataset. The accuracy is different for each model and it is ultimately meant to ensure that with the math used, it can actively use the factors and take each entry into account for the success class that it belongs to. The most significant thing to note from this table is the time it took for the training to finish along with the accuracy scores. The Logistic Regression yielded higher results with more time while the random forest algorithm did the complete opposite.

Figure 3

Classification Report:				
	precision	recall	f1-score	support
High	0.95	0.99	0.97	274
Low	1.00	0.95	0.97	1065
Medium	0.94	0.98	0.96	893
accuracy			0.97	2232
macro avg	0.96	0.97	0.97	2232
weighted avg	0.97	0.97	0.97	2232

Figure 3: provides a classification report of Logistic Regression that derives from Table 2

Figure 3's report gives better insight on the information that comes from Table 2 (for Logistic Regression). The precision, recall, and f1-score columns give high markings to the regression model's testing data. These columns along with the macro and weighted average are tactics that make up the overall accuracy for the model. The support column accounts for the samples of the testing data and it's split through the success class. With this accuracy, it brings inquiries on if the next model (Random Forest) can compare.

Figure 4

Classification Report:				
	precision	recall	f1-score	support
High	0.59	0.69	0.63	274
Low	0.96	0.96	0.96	1065
Medium	0.88	0.83	0.85	893
accuracy			0.87	2232
macro avg	0.81	0.83	0.81	2232
weighted avg	0.88	0.87	0.88	2232

Figure 4: provides a classification report of Random Forest that derives from Table 2

Figure 4's report illustrates the statistics that come from the Random Forest algorithm that was in Table 2. The precision, recall, and f1-score columns are lower than the regression model but it's still fairly consistent among the columns (not including support). The accuracy and average rows in the figure are in the 0.80-0.89 range. It may not be as high or nearly perfect as the logistic regression, but that can be a strength as it has a more realistic approach to active testing and training.

With the comparison of these two models, logistic regression is the better model to use for the analysis of a movie's success. The random forest model can be improved to make it more functional among these datasets and others. The reasoning for the regression model being better is because of the overall average and accuracy that it showed during testing and training. I will take into consideration the fact that the set used only 11,000 samples and around 15 features out

of the 1,000,000+ dataset that existed before preprocessing. However, this sample set is nicer to at least begin on before expanding into a larger database.

V. Conclusion

In conclusion, this project has been a very interesting experience to complete the end of my intro to machine learning class. I felt that this project showcased the best approach to applying the concepts I have learned. The logistic regression algorithm made me feel at peace because of what I have learned over this fall semester, but it was interesting to dive into the random forest algorithm. The best thing about these algorithms is that they were both used for classification with the success classes so it was better to combine the two. However, it was a challenge to settle for a topic that fit with my interests along with finding a dataset that had a large amount of samples and features. Some other datasets had very little samples or plenty of string data types in the .csv file that made it frustrating. I did overcome this eventually with the Kaggle dataset that I found, and the preprocessing approach I implemented made it easier to handle the set. One more challenge I never fully understood was the feature importance portion of the analysis. I see that there were errors during the preprocessing approach and it may be due to text constraints within the dataset. Nevertheless, I still had a great process to learn more about the application of machine learning techniques.

Throughout the process, I learned that there are various features to consider when production companies choose when creating sequels or remakes. I believe that revenue and the vote average are the most important columns in the dataset. The economic status that a

previously released movie can have in decision-making for a sequel is useful to bring the nostalgia back to viewers who have seen the movie in theaters or through other outlets. Another thing I learned was how to actively preprocess data to make it applicable to my research. I started with only dropping some columns to get the hang of it, but I got better when I made the success classes and their scores. Overall, the project improved my understanding of cinematography and analyzed the degree to which a movie can be greenlit for production and released in theaters that can impact the next generation of viewers.

References

Asaniczka, and themoviedb.org. (2025). Full TMDb Movies Dataset 2024 (1M Movies) [Data set]. Kaggle. <https://doi.org/10.34740/KAGGLE/DSV/13942228>

Fithratullah, Muhammad, et al. "OVERLOOKING THE TRANSFORMATION OF VALUES THROUGH THE TRANSNATIONAL AMERICAN REMAKES MOVIES." *Lire Journal (Journal of Linguistics and Literature)* 8.2 (2024): 351-368.7

IMDbPro. "Toy Story (1995)." *IMDb*, <https://www.boxofficemojo.com/title/tt0114709/>.

Accessed 4 Dec. 2025.

Acknowledgement

I, Charles Diggs, acknowledge that I used DeepSeek to assist in completing this final project. It was used in the initial stages mostly to settle on a topic that would be significant to focus on for my report. I asked for various topics until I settled on the degree classification for a film's success.

I also used DeepSeek to add better formatting within my functions to make it easier and readable to those who access my code. Some of the comments in the functions include steps to better explain what the code means.

The last piece that I used the AI for was to explain more of the why to the specific ML algorithms (mainly Random Forest). It was useful in showing the differences between the two and how it was the best choice besides other techniques that wouldn't have been as applicable to my problem statement.

This is a sample text of what I received from the agent that provides the initial stages to choosing the topic of movie success prediction.

For predicting movie success (commonly measured by revenue, profit, ROI, or ratings), you can use a variety of ML algorithms, depending on your prediction goal and data type.

Here's a structured approach:

1. Define the Target Variable

First, decide what “success” means:

- Regression tasks (continuous outcome):
 - Box office revenue
 - Profit (Revenue – Budget)
 - Return on Investment (ROI)
 - IMDb rating or critic score
 - Classification tasks (categorical outcome):
 - Hit vs. Flop (binary)
 - High/Medium/Low success (multiclass)
-

2. Common Algorithms by Task Type

A. Regression (predict revenue/profit)

1. Linear Regression

- Baseline model, assumes linear relationship.
- Good interpretability.
- Weak if relationships are nonlinear.

2. Decision Trees / Random Forest Regressor

- Handles nonlinearities and interactions.

- Random Forest improves accuracy and reduces overfitting.

3. Gradient Boosting Regressors

- XGBoost, LightGBM, CatBoost – often give state-of-the-art results on tabular data.
- Good for capturing complex patterns.

4. Support Vector Regression (SVR)

- Works well with small to medium datasets if tuned properly.
- Can capture nonlinear relationships with kernel tricks.

5. Neural Networks

- Useful if you have a lot of data and possibly combine with other features (text, images).
- Overkill if only using budget, revenue, popularity.

B. Classification (hit vs. flop)

1. Logistic Regression

- Baseline, interpretable.
- Can give probability of success.

2. Random Forest Classifier

- Robust, handles mixed features well.

3. Gradient Boosting Classifiers (XGBoost, etc.)

- Often best performance.

4. Support Vector Machines (SVM)

- Good for clear margin separation in feature space.

5. k-Nearest Neighbors

- Simple, but sensitive to feature scaling and irrelevant features.

3. Feature Considerations

Besides budget, revenue (past?), popularity, consider adding:

- Genre (one-hot encoded)
- Director/actor popularity
- Release season/month
- MPAA rating
- Social media buzz before release
- Competition during release week
- Runtime
- Sequels vs. original

Note: If predicting before release, you cannot use *actual revenue* as a feature; you'd use estimated popularity (trailer views, pre-release sentiment).

4. Special Considerations for Movie Data

- Skewed targets: Movie revenue can span many orders of magnitude → apply log transformation for regression.
- Missing data: Older movies may lack certain metrics.
- Temporal split: Don't randomly split data; use time-based split (train on older movies, test on newer) to avoid temporal leakage.
- Ensemble methods often win competitions (like Netflix Prize, though that was ratings).